

Relazione Progetto Distributed Systems

Gabriel Betti, Giacomo Fornaciari, Giacomo Mariuzza

settembre 2026

Capitolo 1: Introduzione e Contesto Operativo

1.1 Il Paradigma del Federated Learning in Ambito Sanitario (Cross-Silo)

Ad oggi, l'addestramento di modelli di intelligenza artificiale si scontra spesso con le rigide normative sulla privacy dei dati sanitari. Di solito la soluzione è: spostare gigabyte di dati dai vari ospedali verso un singolo server centrale. Dal punto di vista architetturale, questo approccio client-server classico presenta enormi vulnerabilità: crea un Single Point of Failure (SPOF) sistemico, richiede un'elevata larghezza di banda ed espone i dati grezzi a rischi di esfiltrazione.

Il Federated Learning (FL), specificamente nella sua variante **Cross-Silo**, risolve il problema ribaltando il paradigma logico: invece di muovere i dati verso il modello, muove la computazione verso i dati. Questo si traduce in un cluster di macchine indipendenti (i silos ospedalieri) prive di memoria condivisa (*shared-nothing architecture*). Ogni nodo processa i propri dati localmente e aggiorna il proprio stato. Il problema computazionale si trasforma quindi in un problema classico di sistemi distribuiti: la **sincronizzazione periodica di uno stato globale** (il modello unificato) attraverso il puro scambio asincrono di messaggi (i tensori dei pesi), mantenendo la coerenza del sistema globale pur in assenza di un orologio condiviso.

1.2 Obiettivi del Progetto: Affidabilità Infrastrutturale e Capacità Analitiche

L'obiettivo concreto di questo progetto non risiede nella complessità matematica dell'aggregazione, bensì nella progettazione di un *middleware* distribuito e robusto, capace di orchestrare l'intero ciclo di vita dell'addestramento in un ambiente ostile.

In un ecosistema reale, i guasti non sono un'eccezione, ma la regola. I requisiti infrastrutturali primari del nostro progetto sono stati definiti per affrontare le tre sfide principali dei sistemi distribuiti:

- **Gestione dei Fallimenti Parziali (Partial Failures):** Al contrario dei sistemi centralizzati in cui un guasto ferma l'intera applicazione, in questo cluster un nodo può andare in crash (es. esaurimento memoria GPU) mentre gli altri continuano a operare. L'infrastruttura deve garantire il principio di *Fault Isolation*, rilevando il fallimento locale e autoriparandosi senza innescare errori a cascata sulla rete.
- **Coordinamento e Consenso (Leader Election):** Per unificare il modello, la rete deve accordarsi su quale nodo assumerà il ruolo temporaneo di «Aggregatore». In assenza di un server centrale preposto, è necessario implementare un protocollo di elezione distribuita

che consenta ai nodi, comunicando pariteticamente in una rete non fidata, di raggiungere il consenso.

- **Gestione dell'Asincronia (Tolleranza ai Ritardatari):** Non essendoci un *global clock*, i nodi operano a velocità eterogenee. Il sistema deve gestire gli *Stragglers* (nodi lenti o vittime di partizionamenti di rete silenti) tramite meccanismi asincroni, garantendo la *Liveness* (il sistema deve continuare a progredire) senza compromettere la *Safety* del round corrente.

1.3 Panoramica dell'Architettura Ibrida: La Sinergia tra Erlang/OTP e Python

Per implementare un sistema che rispondesse a queste sfide architetturali, abbiamo optato per una netta separazione delle responsabilità (Control Plane vs. Data Plane) utilizzando un'architettura ibrida basata su **Erlang/OTP** e **Python**.

Erlang è stato impiegato come layer di controllo e orchestrazione distribuita. Nato storicamente per i centralini telefonici, Erlang implementa nativamente l'**Actor Model**: processi isolati che non condividono alcuno stato e comunicano solo tramite message passing.

Python, d'altro canto, funge da engine computazionale. Eseguito e strettamente confinato dal supervisore Erlang come un daemon in background locale, lo script Python rimane in attesa di messaggi. Quando l'orchestrazione distribuita lo richiede, Python esegue i calcoli tensoriali, per poi restituire il risultato alla macchina a stati Erlang.

Questa dualità ci permette di incapsulare l'inaffidabilità presente nei calcoli intensivi di Machine Learning (Python) all'interno di una «corazza» puramente orientata alla tolleranza ai guasti e al routing distribuito (Erlang/OTP).

Capitolo 2: Analisi delle Alternative e Scelte Architettureali

In un sistema distribuito, ogni decisione architetturale rappresenta un *trade-off* tra prestazioni, coerenza, disponibilità e complessità. In questo capitolo analizziamo le biforcazioni progettuali affrontate durante lo sviluppo e le ragioni teorico-pratiche che hanno guidato le nostre scelte, alcune delle quali maturate a seguito di test empirici su *Proof of Concept* (PoC) preliminari.

2.1 Comunicazione Inter-Linguaggio: Standard I/O (Ports) vs. NIFs (Native Implemented Functions)

Il primo snodo architetturale ha riguardato l'accoppiamento tra il livello di orchestrazione (Erlang) e il livello di calcolo (Python). La libreria standard di Erlang offre due macro-approcci per interfacciarsi con codice esterno: le NIFs (Native Implemented Functions) o le Ports.

- **Il PoC scartato (NIFs):** Inizialmente, abbiamo sviluppato un modulo sperimentale basato su NIFs, le quali permettono di eseguire codice C/C++ (e tramite binding, Python) direttamente all'interno dello spazio di memoria della BEAM VM. I test di benchmark mostravano prestazioni estreme, eliminando totalmente l'overhead di serializzazione. Tuttavia, durante le simulazioni di stress test (Chaos Monkey), la teoria dei Sistemi Distribuiti si è scontrata con la realtà: poiché le NIFs condividono lo spazio di indirizzamento con la Virtual Machine, un errore critico lato Python (nello specifico, abbiamo simulato un errore *Out of Memory* (OOM) e un *Segmentation Fault* generato da tensori malformati)

ha provocato il crash incontrollabile dell'intera istanza Erlang. Questo violava ovviamente il principio fondante della *Fault Isolation*: un banale errore matematico locale abbatteva l'intero nodo di rete, distruggendo la topologia e innescando *timeout* a catena su tutto il cluster.

- **La scelta architetturale (Erlang Ports via Standard I/O):** A seguito dei fallimenti del PoC, abbiamo ripiegato definitivamente sull'utilizzo di `open_port/2`, che delega l'esecuzione dello script Python a un processo figlio del sistema operativo indipendente (OS process). La comunicazione avviene esclusivamente tramite flussi di Standard I/O (`stdin/stdout`). Questo approccio estende il paradigma *shared-nothing* di Erlang anche all'ambiente computazionale. Se il processo Python va in crash critico, la BEAM VM rimane fisicamente e logicamente intatta; intercetta semplicemente il segnale di chiusura della porta (pipe rotta) e innesca la propria strategia di *Supervisione OTP*, riavviando un nuovo processo pulito in pochi millisecondi senza che il cluster di rete ne risenta.

2.2 Algoritmi di Consenso: Bully Algorithm vs. Approcci basati su Quorum (Raft/Paxos)

Per orchestrare il round di Federated Learning, il cluster *Full-Mesh* necessita di eleggere temporaneamente un nodo «Leader» (Aggregatore), superando l'assenza di un server centrale preposto. Abbiamo valutato i principali algoritmi di consenso distribuito.

- **L'alternativa teorica (Raft / Paxos):** Algoritmi come Raft o Paxos sono lo standard industriale per la replicazione di macchine a stati (*State Machine Replication*). Basandosi su log replicati e maggioranze (Quorum), garantiscono *Strong Consistency* in reti asincrone soggette a partizionamenti. Tuttavia, pur essendo lo stato dell'arte, risultavano troppo onerosi (*overkill*) per il nostro dominio applicativo. Il Federated Learning Cross-Silo non richiede di mantenere un log storicizzato e persistente di chi è stato il leader in passato; richiede unicamente un accordo puntuale ed effimero per coordinare il calcolo asincrono corrente. L'overhead di messaggistica continua e la gestione dello stato su disco richiesta da Raft avrebbero introdotto una latenza strutturale in antitesi con i nostri requisiti.
- **La scelta architetturale (Bully Algorithm):** Abbiamo implementato una variante pura e reattiva del **Bully Algorithm**. In reti di dimensioni contenute (Cross-Silo) e tendenzialmente sincrone/semi-sincrone, questo algoritmo totalmente *stateless* eccelle per semplicità. Il Bully sfrutta una gerarchia di rete preesistente: nel nostro caso, abbiamo evitato di cablare ID artificiali e abbiamo utilizzato direttamente la comparazione nativa della macchina virtuale (`node()`), poiché in Erlang la disuguaglianza tra atomi (es. `siloc@host > silob@host`) è deterministica e immutabile. Questa scelta comporta un noto *trade-off*: l'*Avalanche Effect* (generazione ridondante di messaggi $O(N^2)$ nel caso di failure simultanee). Tuttavia, nei nostri collaudi in topologie Cross-Silo ristrette, il traffico di rete generato durante l'elezione si è dimostrato ampiamente trascurabile.

2.3 Gestione delle Dipendenze: L'Approccio «Zero-Dependencies» e la Serializzazione Custom

La trasmissione asincrona dei pesi matematici solleva il problema della serializzazione dei messaggi cross-language. Erlang deve formattare e inviare un payload che Python possa comprendere nativamente (JSON).

- **L'esperimento scartato (Librerie Esterne JSON):** In un test intermedio avevamo importato `jsx`, una delle librerie Erlang standard per la decodifica/codifica JSON. Sebbene il codice risultasse più leggibile, ci siamo scontrati con il problema strutturale delle dipendenze: il progetto non era più *self-contained*. Richiedeva l'uso obbligatorio di tool di build avanzati (`rebar3`), esponeva il progetto a vulnerabilità esterne (l'aumento del *failure domain*) e creava attrito per il deployment su nodi isolati (*air-gapped*) o non dotati di strumenti di compilazione moderni.
- **La scelta architetturale (Zero-Dependencies e Parsing Vanilla):** Dal punto di vista dei Sistemi Distribuiti, il *middleware* (Erlang) non deve necessariamente elaborare o validare semanticamente il payload, ma solo instradarlo. Abbiamo quindi trattato i tensori matematici in transito come stringhe opache (*Opaque Payloads*). Sfruttando le funzioni stringa basilari della standard library OTP (`lists:splitwith/2` per intercettare il flag direzionale "`AGGREGATE|...`" e `string:join/2` per concatenare la lista dei risultati distribuiti), Erlang assembla empiricamente la struttura testuale di un JSON Array multidimensionale. La vera e costosa deserializzazione (da stringa JSON a float) viene confinata ed eseguita esclusivamente a valle, all'interno del nodo Python. Questa soluzione «artigianale» ha permesso di ottenere un'applicazione totalmente *Zero-Dependencies*, compilabile nativamente in pochi istanti tramite `erlc`.

Capitolo 3: Architettura a Livello di Nodo (Node-Level & Fault Isolation)

Un assioma fondamentale dei sistemi distribuiti impone che l'infrastruttura di rete non debba mai collassare a causa di un'anomalia locale. L'obiettivo primario di questa prima fase di sviluppo (Node-Level) è stato quello di consolidare l'architettura del singolo silo ospedaliero, applicando rigorosamente il principio di *Fault Isolation*. Abbiamo strutturato il nodo affinché l'ambiente computazionale (Python) fosse completamente disaccoppiato dalle logiche di orchestrazione (Erlang), proteggendo quest'ultimo da eventuali instabilità del calcolo tensoriale.

3.1 Scaffolding dell'Infrastruttura secondo i Canoni OTP (Applicazioni e Metadati)

Il punto di partenza ha richiesto l'abbandono di script «volanti» (tipici dei Proof of Concept) in favore di un'infrastruttura formale basata sui canoni **OTP (Open Telecom Platform)**. OTP non è solo un framework, ma un insieme di design pattern architetturali per la costruzione di sistemi tolleranti ai guasti.

Abbiamo ingegnerizzato il repository separando nettamente i domini di competenza:

- `erlang_orchestrator/src/`: Directory dedicata al codice sorgente Erlang puro (`.erl`), contenente le definizioni dei comportamenti (es. `gen_server`, `supervisor`).
- `erlang_orchestrator/ebin/`: Directory di output per i file binari precompilati (`.beam`) e, soprattutto, per il file di metadati dell'applicazione (`.app`).
- `python_worker/`: Sandbox isolata destinata a ospitare gli script di Machine Learning.

L'adozione di questa gerarchia formale ci ha permesso di trattare il nodo non come un semplice script in esecuzione, ma come un'**Applicazione OTP** autocontenuta (`application:start/1`),

dotata di un proprio ciclo di vita, di variabili d'ambiente isolate e pronta per essere pacchettizzata e distribuita su server bare-metal o containerizzati.

3.2 Il Supervision Tree: Strategia One-for-One e Disaccoppiamento dei Percorsi

Il pilastro della resilienza in Erlang è l'Albero di Supervisione (*Supervision Tree*). Abbiamo implementato il modulo `orchestrator_sup.erl`, un supervisore di root progettato per monitorare i processi critici del nodo.

In questa fase, il supervisore è stato configurato con una strategia di riavvio **one_for_one**: se un processo figlio «muore», solo quel processo viene riavviato, lasciando intatti eventuali altri servizi concorrenti. Il primo «figlio» (worker) inserito nell'albero è stato il `python_worker_srv.erl`, un `gen_server` dedicato a incapsulare la porta di comunicazione con il sistema operativo.

Per garantire la portabilità del nodo in ambienti eterogenei (evitando il collasso dovuto a percorsi file non validi), abbiamo rimosso ogni path assoluto *hardcoded* all'interno del server. Il `gen_server` interroga dinamicamente i metadati OTP tramite `application:get_env(erlang_orchestrator, python_worker_script, DefaultPath)`. Questo disaccoppiamento spaziale permette di lanciare il nodo su macchine virtuali diverse iniettando semplicemente il path corretto in fase di avvio, senza richiedere alcuna ricompilazione del modulo Erlang.

3.3 Il Bridge Asincrono Erlang-Python e la Macchina a Stati Locale

Una volta garantita la supervisione, abbiamo stabilito il circuito di comunicazione tra l'Actor Model (Erlang) e il demone computazionale (Python). Come stabilito in fase di analisi (Capitolo 2.1), il bridge si basa sull'invio di messaggi testuali asincroni tramite flussi di Standard I/O.

Lato Python (`worker.py`), abbiamo superato il concetto di script imperativo a esecuzione singola, implementando una vera e propria **macchina a stati reattiva**. Il programma entra in un loop di ascolto perpetuo (`sys.stdin.readline()`). Quando Erlang immette un payload nel buffer di sistema, Python si risveglia, esegue il parsing del comando, simula l'elaborazione del tensore e inietta il risultato nello Standard Output.

Due accorgimenti tecnici si sono rivelati cruciali in questa fase:

1. **Svuotamento dei Buffer OS:** L'uso esplicito di `sys.stdout.flush()` è stato reso obbligatorio. Senza di esso, i sistemi operativi tendono a bufferizzare l'output per ottimizzare le risorse, causando *deadlock* distribuiti in cui Erlang rimane in attesa infinita di una risposta già calcolata ma non ancora fisicamente trasmessa dalla pipe.
2. **Defensive Programming:** Abbiamo inserito blocchi `try/except ValueError` attorno al parser logico. Se un messaggio arriva corrotto dalla rete, Python intercetta l'eccezione e logga l'errore senza andare in crash, evitando di innescare inutilmente l'albero di supervisione per colpa di pacchetti malformati.

3.4 Validazione della Fault Isolation: Esecuzione del «Chaos Monkey Test»

Per dimostrare che il nodo fosse a prova di crash e che il principio di *Fault Isolation* fosse stato implementato correttamente, abbiamo sottoposto l'architettura a uno stress test di natura distruttiva, noto nell'ingegneria del software come «Chaos Monkey Test».

Abbiamo avviato il cluster locale ed effettuato richieste valide (es. l'invio di un payload «21» per ottenere l'operazione matematica attesa, ricevendo correttamente «42.0»). Successivamente, abbiamo simulato un fallimento hardware critico (es. esaurimento improvviso della VRAM) interagendo direttamente col sistema operativo: tramite il comando Linux `ps aux`, abbiamo individuato il Process ID (PID) del worker Python in background e lo abbiamo terminato brutalmente con un segnale di sistema `SIGKILL`.

L'esito del test: Nessun kernel panic e nessun congelamento del terminale Erlang. La BEAM VM ha intercettato il *broken pipe* della porta standard I/O. Il `gen_server` locale è terminato in modo anomalo, il che ha immediatamente attivato il trigger del supervisore `orchestrator_sup`. Grazie alla configurazione `restart_type: permanent`, il supervisore ha resuscitato il processo figlio Erlang il quale, a sua volta, ha istanziato un nuovo processo OS Python.

L'intera procedura di autoriparazione si è conclusa in poche frazioni di millisecondo. Inviando un nuovo comando di calcolo al nodo un istante dopo l'omicidio del processo, il sistema ha risposto regolarmente, dimostrando una tolleranza ai fallimenti parziali impeccabile.

Capitolo 4: Architettura di Rete e Topologia (Network-Level)

Dopo aver blindato il singolo nodo garantendone l'isolamento dai guasti locali, lo step architetturale successivo è stato l'interconnessione dei silos ospedalieri. L'obiettivo del livello di rete (Network-Level) è instaurare una topologia di comunicazione distribuita e decentralizzata, capace di autoconfigurarsi senza l'intervento manuale di un operatore di sistema.

4.1 Dinamiche di Rete Distribuite: Il Ruolo EPMD (Erlang Port Mapper Daemon)

Per la comunicazione inter-nodo abbiamo sfruttato l'infrastruttura di distribuzione nativa della BEAM VM. Alla base di questo strato di rete risiede l'**EPMD (Erlang Port Mapper Daemon)**, un demone vitale che funge da *Name Server* per l'ambiente distribuito.

Nei sistemi distribuiti classici, la comunicazione richiede la gestione esplicita di indirizzi IP e porte TCP. EPMD astrae questa complessità fornendo *Location Transparency*: mappa dinamicamente i nomi simbolici dei nodi (es. `siloa@hostname`) sulle porte fisiche allocate dal sistema operativo.

Sfruttando EPMD, abbiamo optato per una topologia di rete **Full-Mesh** (a maglia completa). Se in contesti *Cross-Device* (con milioni di smartphone) una maglia completa collapserebbe sotto il peso delle connessioni $O(N^2)$, nel nostro scenario *Cross-Silo* (dove il numero di istituti di ricerca o ospedali è limitato a poche decine) la topologia Full-Mesh risulta la scelta ideale: garantisce massima ridondanza, latenza minima (essendo ogni nodo a un solo «hop» di distanza dagli altri) e l'assenza di router centrali vulnerabili (SPOF).

4.2 Modulo di Auto-Discovery e Inizializzazione Asincrona del Cluster

Per evitare il fragile e tedioso processo di accoppiamento manuale dei nodi, abbiamo ingegnerizzato un modulo dedicato alla scoperta automatica dei peer: il `cluster_manager_srv.erl`.

Questo `gen_server` è stato registrato come primo «figlio» (con strategia `permanent`) all'interno dell'albero di supervisione (`orchestrator_sup.erl`), garantendo che il sottosistema di rete si attivi prima dei worker di calcolo.

La sfida architetturale principale nella fase di *discovery* è la gestione dell'asincronia: le chiamate di rete sono per definizione bloccanti e imprevedibili (a causa di latenze o nodi spenti). Se avessimo inserito il tentativo di connessione direttamente nella funzione di callback `init/1` del modulo, un ritardo di rete avrebbe «congelato» l'intero albero di supervisione, impedendo al nodo di avviarsi.

Per risolvere questa potenziale *race condition*, abbiamo disaccoppiato l'avvio sfruttando il message passing asincrono: all'interno della `init/1`, il server invia un messaggio a se stesso (`self() ! discover_nodes`) e conclude immediatamente l'inizializzazione. Solo in un secondo momento, elaborando la propria casella di posta, il server estrae dinamicamente la lista dei nodi obiettivo (i *seed nodes*) tramite `application:get_env` ed esegue il ping di rete (`net_adm:ping/1`), stampando a console feedback visivi diagnostici (es. «✓» per il successo e «▲» per *host unreachable*), il tutto senza mai bloccare il thread principale della macchina virtuale.

4.3 Risoluzione delle Anomalie di Build (Gestione file .app.src)

Durante lo sviluppo e il deployment del cluster su terminali eterogenei, ci siamo scontrati con un'anomalia subdola che impediva il corretto caricamento dell'infrastruttura di rete. L'invocazione di `application:start(erlang_orchestrator)` falliva silenziosamente, impedendo l'istanza del supervisore e dei suoi servizi dipendenti (incluso il modulo di Auto-Discovery).

L'analisi del problema ha rivelato una discrepanza nel processo di build. Il compilatore nativo `erlc` è progettato per tradurre i file sorgente (`.erl`) in binari (`.beam`), ma non gestisce in autonomia la copia dei file di metadati. L'assenza del file descrittore dell'applicazione (`.app`) nella cartella di destinazione (`ebin/`) impediva alla Virtual Machine di esporre le proprietà del modulo e le variabili d'ambiente.

Invece di ricorrere a complessi tool esterni di release generation, la risoluzione architetturale è consistita nell'introduzione di uno step esplicito nella *build-pipeline* (tramite comandi shell integrati) per forzare la copia e ridenominazione del file `src/erlang_orchestrator.app.src` in `ebin/erlang_orchestrator.app`. Questa correzione chirurgica ha sbloccato definitivamente la corretta lettura delle proprietà d'ambiente, garantendo un *bootstrap* deterministico su tutti i nodi.

4.4 Collaudo Operativo della Topologia Full-Mesh e Transitività

La verifica dell'infrastruttura di rete è stata condotta simulando un cluster di tre silos sanitari indipendenti (`siloa`, `silob`, `siloc`). L'autenticazione e la sicurezza del cluster sono state garantite dalla condivisione di un *magic cookie* crittografico uniforme (`-setcookie federated_cookie`).

La sequenza di collaudo ha dimostrato in modo inequivocabile la proprietà di **Transitività Distribuita** del protocollo Erlang:

1. **Silo A:** Avviato in totale isolamento, senza alcun nodo *seed* configurato.
2. **Silo B:** Avviato fornendo esclusivamente il Silo A come *seed node*. La console del Silo B ha notificato con successo l'avvenuta connessione bidirezionale al Silo A.
3. **Silo C:** Avviato fornendo *esclusivamente* il Silo B come *seed node*, ignorando totalmente l'esistenza del Silo A.

All'avvio del terzo nodo, il sistema ha agganciato correttamente il Silo B. Tuttavia, grazie alla mediazione automatica dell'EPMD, il Silo B ha informato il Silo C della presenza del Silo A. In frazioni di secondo, senza alcun intervento manuale, i socket TCP si sono riconfigurati, chiudendo magicamente la maglia.

Interrogando la funzione di sistema `nodes()` dal terminale del Silo C, il cluster ha risposto restituendo l'array `['silob@host', 'siloa@host']`, dimostrando empiricamente il raggiungimento di una topologia Full-Mesh perfetta, stabile e pronta per ospitare l'algoritmo di elezione del Leader.

Capitolo 5: Elezione del Leader e Consenso Distribuito

Con l'instaurazione di una topologia di rete Full-Mesh stabile e reattiva, il cluster si presenta come un insieme di nodi paritetici (peer-to-peer). Tuttavia, per orchestrare le fasi di un algoritmo deterministico come il Federated Learning, è necessario superare questa simmetria.

5.1 La Necessità di un Nodo «Aggregatore Globale» nel Federated Learning

Nei sistemi di Federated Learning tradizionali, architettati secondo il modello client-server, esiste un server centrale fisso (*Parameter Server*) che si occupa di dettare i tempi dei round di addestramento, raccogliere i gradienti ed eseguire la media matematica.

Nel nostro dominio decentralizzato Cross-Silo, l'assenza di un server preposto rappresenta sia il principale vantaggio in termini di *Fault Tolerance* (assenza di SPOF), sia la principale sfida algoritmica. Affinché il calcolo globale (FedAvg) possa avvenire senza causare collisioni di stato o *split-brain* (dove più nodi credono di dover fare l'aggregazione simultaneamente), la rete deve eleggere in modo dinamico un nodo temporaneo a cui demandare il ruolo di «Aggregatore Globale» (Leader). Questo richiede un algoritmo di consenso distribuito in grado di convergere verso una decisione unanime, persino in presenza di guasti o di nodi irraggiungibili.

5.2 Implementazione del Bully Algorithm tramite Comparazione Nativa degli Atomi

Come argomentato in sede di analisi architetturale (Cap. 2.2), la scelta è ricaduta sull'implementazione del **Bully Algorithm**. Per gestire questa logica, abbiamo isolato il dominio del consenso introducendo un nuovo microservizio OTP, il `bully_srv.erl`, un `gen_server` dedicato esclusivamente a governare la macchina a stati dell'elezione e integrato nel *Supervision Tree* con strategia di riavvio `permanent`.

La sfida principale nell'implementazione di algoritmi gerarchici come il Bully è l'assegnazione di identificatori numerici ai nodi per stabilirne la priorità. Mappare dinamicamente ID interi a nodi effimeri in una rete distribuita richiederebbe a sua volta un protocollo di sincronizzazione dello stato, introducendo un pericoloso paradosso del consenso.

Abbiamo risolto questo limite ingegneristico sfruttando una peculiarità intrinseca della BEAM VM: la **comparazione nativa e deterministica degli atomi**. In Erlang, la funzione `node()` restituisce il nome simbolico del nodo (es. `'siloc@hostname'`). Poiché il linguaggio garantisce un ordinamento lessicografico rigido e immutabile per gli atomi, l'operatore relazionale matematico (`>`) è sufficiente per stabilire una gerarchia globale coerente: `'siloc@host'` è universalmente e intrinsecamente maggiore di `'silob@host'`, fornendo al cluster una priorità di elezione «gratuita» e *stateless*, senza necessità di cablare configurazioni aggiuntive.

Inoltre, per evitare *race condition* catastrofiche durante la fase di *Auto-Discovery* (Cap. 4), il modulo è stato progettato per disaccoppiare l’inizializzazione del demone dall’innesco dell’elezione: la funzione `init/1` alloca unicamente lo stato neutro (Leader ignoto, nessun timer attivo), demandando l’inizio della contesa a un’invocazione esplicita tramite l’API `bully_srv:start_election/0`.

5.3 La Macchina a Stati Inter-Nodo: Messaggistica ELECTION, ALIVE e COORDINATOR

La convergenza dell’algoritmo è stata mappata su un set di tre messaggi scambiati in modo puramente asincrono (*non-blocking*) tramite chiamate `gen_server:cast/2`. Il protocollo si sviluppa rigorosamente secondo le seguenti regole di transizione di stato:

1. **ELECTION** (`{election, FromNode}`): Quando un nodo innesca un’elezione, invia questo messaggio in broadcast *esclusivamente* ai nodi con identificativo strettamente maggiore del proprio. Contestualmente, avvia un timer di attesa locale (2000 ms).
2. **ALIVE** (`{alive, FromNode}`): Se un nodo riceve una **ELECTION** da un peer gerarchicamente inferiore, risponde istantaneamente con un messaggio **ALIVE**. Questo messaggio agisce come segnale di soppressione (*veto*): il nodo inferiore riceve l’**ALIVE**, cancella il proprio timer e cede il passo. Simultaneamente, il nodo superiore avvia una propria elezione verso l’alto (se non è già in corso).
3. **COORDINATOR** (`{coordinator, LeaderNode}`): Se il timer di elezione (2000 ms) scade senza che il nodo abbia ricevuto alcun messaggio **ALIVE**, significa matematicamente che esso è il nodo di grado massimo in quel momento raggiungibile nella maglia (tutti i nodi superiori sono teoricamente guasti o inesistenti). Il nodo si autoproclama Leader, aggiorna il proprio stato locale e trasmette il messaggio **COORDINATOR** a tutti gli altri membri del cluster, imponendo la propria leadership.

5.4 Analisi del Comportamento di Rete: Tolleranza ai Guasti e «Avalanche Effect»

L’infrastruttura di consenso è stata collaudata sul cluster di tre nodi, innescando deliberatamente l’elezione dal gradino gerarchico più basso (`siloa`). L’osservazione dei log distribuiti ha confermato l’esatta esecuzione teorica dell’algoritmo:

1. `siloa` ha inviato **ELECTION** sfidando `silob` e `siloc`.
2. `silob` ha soppresso `siloa` con un **ALIVE** e ha sfidato `siloc`.
3. `siloc`, essendo il vertice della topologia, ha soppresso sia `siloa` che `silob`.
4. Allo scadere esatto del timeout, `siloc` ha emesso il dictat **COORDINATOR**, aggiornando lo stato globale in modo unanime.

Durante i test sotto stress, è emerso un comportamento architetturale degno di nota accademica: l’**Avalanche Effect** (Effetto Valanga). I log del cluster hanno evidenziato la ricezione occasionale di messaggi **COORDINATOR** duplicati da parte del Leader neoeletto. Dal punto di vista della teoria dei Sistemi Distribuiti, questo non costituisce un’anomalia applicativa (*bug*), ma la dimostrazione tangibile dell’aderenza rigorosa al design pattern puro del Bully Algorithm

in presenza di asincronia spinta. L'effetto valanga si innesca poiché il nodo maggiore (**siloc**), ricevendo sfide **ELECTION** quasi simultanee da molteplici nodi inferiori, è spinto a ri-allocare o gestire timer concorrenti che, a scadenza, generano broadcast di vittoria ridondanti.

Essendo il nostro un ambiente Cross-Silo (con topologia N circoscritta), l'aggiunta di logica *stateful* per filtrare la valanga avrebbe appesantito inutilmente il modulo. Il cluster tollera e smaltisce questa ridondanza in modo nativo, privilegiando la semplicità algoritmica e la *liveness* del sistema rispetto a un'ottimizzazione microscopica del traffico di rete.

Capitolo 6: Orchestrazione e Calcolo Globale (Federated Averaging)

Risolti i problemi infrastrutturali di base (topologia di rete e consenso), il focus architetturale si sposta sul dominio applicativo: l'esecuzione concreta dell'algoritmo di *Federated Learning*. In questo capitolo analizziamo come l'ecosistema ibrido coordina il calcolo tensoriale distribuito, implementando l'orchestrazione del *Federated Averaging* (*FedAvg*) in modo rigorosamente asincrono e non bloccante.

6.1 Il Ciclo di Vita del Round di Addestramento Asincrono

Nel Federated Learning, il progresso del modello globale è scandito in epoche o «Round». Il coordinamento di questi cicli è affidato al microservizio OTP `fl_manager_srv.erl`.

Una volta che il *Bully Algorithm* ha designato l'Aggregatore Globale, quest'ultimo ha l'onere di innescare il calcolo. L'inizializzazione del round (tramite l'API `start_round/0`) avviene attraverso un'operazione di *broadcast asincrono*: il Leader itera sulla topologia dei nodi attivi (`nodes() ++ [node()]`) e invia a ciascuno un messaggio del tipo `{train_command, LeaderNode}`.

Dal punto di vista dei Sistemi Distribuiti, questo approccio *fire-and-forget* (implementato tramite `gen_server:cast/2`) è cruciale: il Leader impartisce l'ordine a tutta la rete (incluso se stesso) senza mettersi in attesa sincrona (*polling* o *blocking call*) della risposta di alcun peer. Questo permette all'Aggregatore di tornare istantaneamente in stato di riposo (*idle*), pronto a reagire all'arrivo dei primi risultati o alla gestione di eventuali anomalie di rete, massimizzando la *liveness* del sistema.

6.2 Routing Intelligente e Gestione Non Bloccante nella BEAM VM

Alla ricezione del comando di addestramento, il manager locale di ogni silo deve delegare il lavoro intensivo all'engine computazionale (Python). Qui entra in gioco il design pattern del **Reactor** implementato nella BEAM VM.

Il modulo bridge (`python_worker_srv.erl`) comunica con Python inoltrando il comando nello *Standard Input* del processo OS. Poiché il calcolo di una rete neurale può richiedere da pochi secondi a diverse ore, Erlang non deve mai attendere attivamente la fine dell'elaborazione.

Il bridge è stato ingegnerizzato per intercettare l'output di Python (iniettato nello *Standard Output*) in modo reattivo, attraverso la callback `handle_info` che cattura la tupla di sistema `{Port, {data, {eol, Line}}}`. Invece di eseguire computazioni pesanti su questa stringa, il bridge si limita a fare *routing intelligente*: esegue immediatamente un `gen_server:cast(fl_manager_srv, {python_result, Line})`. Questo disaccoppiamento garantisce che il thread

responsabile della porta OS sia sempre libero di svuotare il buffer, delegando il processamento logico del risultato al manager del Federated Learning, nel pieno rispetto dell'Actor Model.

6.3 L'Algoritmo FedAvg: Calcolo Tensoriale Simulato e Fusione dei Pesi

Il cuore matematico del sistema risiede nel Data Plane (Python). Lo script `worker.py` è stato ingegnerizzato come una **macchina a stati reattiva**, il cui comportamento è governato dai prefissi testuali inviati da Erlang. Il ciclo di calcolo prevede due stati fondamentali:

1. **Fase di Addestramento Locale (TRAIN):** Alla ricezione di questo trigger, il nodo Python simula l'addestramento locale su dati clinici proprietari. In questa implementazione formale, il nodo genera un vettore di pesi randomizzati (es. `[0.85, 1.12, 0.91]`) per emulare la naturale varianza statistica (Non-IID) dei dati sanitari distribuiti, e lo restituisce alla VM con il prefisso `TRAIN_RES|`.
2. **Fase di Fusione Globale (AGGREGATE|):** Quando il Leader raccoglie tutti i contributi, deve unificarli. Invia al proprio worker Python locale il prefisso `AGGREGATE|` seguito da un array multidimensionale (una lista contenente i vettori di tutti i silos). Python effettua il parsing della struttura, calcola la media aritmetica colonna per colonna (che costituisce l'essenza dell'algoritmo *Federated Averaging* puro) e restituisce il modello globale unificato con il flag `AGGREGATE_RES|`.

6.4 Serializzazione Dinamica e Formattazione Bidirezionale (JSON-Vanilla)

L'integrazione fluida tra il Control Plane (Erlang) e il Data Plane (Python) impone la necessità di un protocollo di serializzazione strutturato (JSON), ma coerentemente con l'approccio *Zero-Dependencies* illustrato in precedenza (Cap. 2.3), abbiamo optato per una manipolazione testuale *vanilla*.

Per processare i messaggi in ingresso, Erlang utilizza il pattern matching nativo abbinato alla funzione `lists:splitwith/2`. Questa operazione separa chirurgicamente l'intestazione testuale (es. `TRAIN_RES|`) dal *payload* numerico puro, iterando fino al carattere separatore (`|`). Questo evita il ricorso a pesanti librerie di Espressioni Regolari (Regex), mantenendo altissima l'efficienza ciclica della VM.

La vera ingegnosità risiede però nell'accumulo per l'aggregazione. Il Leader istanzia un contatore dinamico (`expected_nodes`) calcolato in base all'ampiezza istantanea del cluster. Quando il quorum viene raggiunto, l'accumulatore interno (`accumulated_weights`) contiene una lista di stringhe formattate. Invece di decodificare e ricodificare i tensori, il Leader Erlang costruisce empiricamente un array JSON valido concatenando le stringhe opache con una semplice virgola (`string:join(NewAcc, ",")`) e racchiudendole tra parentesi quadre (`"[" ++ ... ++ "]"`).

Questa serializzazione dinamica a basso costo computazionale permette a Erlang di inviare a Python una struttura complessa perfettamente valida, che verrà infine processata dalla libreria nativa `json.loads()` dell'engine computazionale. Il round si conclude visualizzando a console il nuovo modello globale unificato (es. `🔴 [FED-AVG] Round completato!`), confermando l'assoluta affidabilità del circuito bidirezionale asincrono.

Capitolo 7: Resilienza Avanzata e Tolleranza ai Ritardatari (Stragglers)

L'adozione di un'architettura distribuita risolve il problema del *Single Point of Failure*, ma introduce inevitabilmente lo spettro dell'asincronia imprevedibile. In questo capitolo analizziamo l'identificazione e la risoluzione di una delle vulnerabilità più critiche nei sistemi distribuiti sincroni o semi-sincroni: la dipendenza bloccante dai nodi periferici.

7.1 Il Collo di Bottiglia del Sincronismo: Il Problema degli Stragglers nel FL

Nel Federated Learning teorico, l'aggregazione globale dei pesi si aspetta la risposta di tutti i partecipanti per chiudere un'epoca di addestramento. Nell'infrastruttura originaria, il nodo Leader accumulava passivamente i risultati: calcolava il numero di nodi attesi (`expected_nodes`) e rimaneva in uno stato di attesa indefinita finché la lista dei pesi ricevuti non uguagliava tale valore.

In un ambiente reale (ospedali con capacità computazionali disomogenee, hardware obsoleto o reti instabili), questa attesa passiva rappresenta un *collo di bottiglia del sincronismo* fatale. Un singolo nodo lento (definito in letteratura come *Straggler*) o un nodo vittima di un partizionamento di rete silente (dove il socket TCP rimane aperto ma il processo applicativo si blocca) può provocare la paralisi dell'intero round globale, compromettendo la *Liveness* (progresso) dell'intero sistema pur di garantire la *Safety* (completezza dei dati).

7.2 Implementazione del Timeout Asincrono e Aggregazione Parziale d'Emergenza

Per mitigare questa vulnerabilità architetturale senza ricorrere a costosi e inefficienti meccanismi di *polling* continuo, abbiamo sfruttato i timer asincroni nativi della BEAM VM. La logica del `fl_manager_srv` è stata potenziata per gestire l'assenza temporale come un evento computazionale.

1. **Allocazione del Timer:** All'innesco del round (`start_round`), il Leader determina il quorum necessario e contestualmente avvia un timer in background invocando `erlang:send_after(5000, self(), round_timeout)`. Questo alloca un riferimento (`TimerRef`) nello stato del processo.
2. **Successo Nominale (Happy Path):** Se tutti i nodi rispondono entro la finestra tollerata di 5000 ms, l'accumulatore raggiunge il quorum. Il sistema, prima di eseguire il *Federated Averaging*, cancella proattivamente il timer (`erlang:cancel_timer(TimerRef)`), scongiurando *memory leak* o eventi ritardati, e azzera la variabile di stato.
3. **Aggregazione Parziale d'Emergenza:** Se un nodo non risponde, il timer scade e la VM inietta il messaggio asincrono `round_timeout` nella mailbox del manager. Il sistema intercetta il messaggio tramite la callback `handle_info`, valuta i pesi accumulati fino a quel momento (ignora i nodi silenti) e forza un'**aggregazione d'emergenza parziale**. Questo *trade-off* ingegneristico sacrifica una frazione di accuratezza del modello (dovuta ai dati mancanti) per garantire la resilienza e la continuità operativa del cluster globale.

7.3 La Resilienza Intrinseca di Erlang: Analisi dei Fallimenti Simulati (Crash ed Esclusione)

Il collaudo di questa logica di tolleranza ai guasti ha richiesto un approccio iterativo, svelando un aspetto affascinante: la resilienza intrinseca dell'ecosistema Erlang/OTP è tale da rendere attivamente «difficile» simulare un guasto persistente che inneschi il timeout.

- **Ostacolo 1 (L'Albero di Supervisione):** Nel primo test, abbiamo simulato un guasto hardware spegnendo brutalmente il worker Python di un nodo secondario (`gen_server:stop(python_worker_srv)`). Il timeout del Leader non è mai scattato. Il Supervisore locale, configurato con `restart_type: permanent`, ha intercettato l'uscita prematura e ha ricreato il processo in una frazione di millisecondo. Il nodo è riuscito a processare la richiesta in tempo, annullando il timer del Leader. L'infrastruttura si è auto-riparata prima che il livello applicativo se ne accorgesse.
- **Ostacolo 2 (Topologia di Rete Dinamica):** Nel secondo test, abbiamo provato a far «cadere» per intero un silo, arrestando la sua Virtual Machine (`init:stop()`). Anche in questo caso, il timeout non è scattato. Il demone di rete (`epmd`) del Leader ha rilevato istantaneamente la caduta del socket TCP e ha rimosso il nodo dalla topologia. Di conseguenza, all'avvio del round, il Leader ha ricalcolato dinamicamente il quorum basandosi sulla funzione `nodes()`, abbassando i nodi attesi (da 3 a 2) e concludendo immediatamente il round con i soli nodi superstiti, senza incorrere in alcun ritardo.

7.4 Il Test Definitivo: Simulazione del «CPU Lock» (Partizionamento Silente)

L'unico modo per validare empiricamente l'implementazione del timeout è stato riprodurre la condizione più insidiosa per un sistema distribuito: il *Partizionamento Silente* o «CPU Lock».

Tramite la funzione di sistema `sys:suspend(fl_manager_srv)`, abbiamo congelato attivamente lo stato del processo manager di un nodo bersaglio. In questo scenario, le sonde di rete di EPMD (che operano a livello di demone VM) hanno continuato a rilevare il nodo come vivo e connesso, inducendo il Leader ad attenderne la risposta (quorum a 3). Tuttavia, il livello applicativo locale era totalmente incapace di processare la coda di messaggi.

Allo scadere esatto dei 5 secondi di silenzio, il Leader ha finalmente catturato l'evento di timeout. Il sistema ha stampato un *warning* diagnostico a console ed ha eseguito l'aggregazione parziale sui due nodi attivi in totale autonomia. Questo test finale ha certificato formalmente che l'architettura è in grado di tollerare non solo i *crash* espliciti e le cadute di rete, ma anche le anomalie bizantine di stallo computazionale periferico, garantendo una robustezza di livello enterprise.

Capitolo 8: Collaudo Formale e Test Automatizzati (Automated Testing)

L'affidabilità di un sistema distribuito, per quanto ben architettata teoricamente, non può prescindere da una validazione empirica e riproducibile. Oltre ai collaudi manuali di rete («Chaos Monkey» e stress test), l'architettura è stata sottoposta a un collaudo formale tramite l'implementazione di test automatizzati, garantendo che le logiche di orchestrazione e *fault tolerance* resistano a future regressioni.

8.1 Strategia di White-Box Testing nel Paradigma Funzionale

Testare un *middleware* distribuito presenta una sfida intrinseca: l'accoppiamento tra il codice applicativo e lo stato della rete/sistema operativo. Avviare intere macchine virtuali, *demoni* OS (Python) e *socket* TCP per verificare la correttezza di una singola funzione decisionale risulta estremamente oneroso (spesso definito come *Integration Testing* pesante).

Tuttavia, il framework OTP e il paradigma funzionale di Erlang offrono una soluzione elegante: le funzioni interne di un `gen_server` (come `handle_cast/2` o `handle_info/2`) sono architeturalmente implementate come **funzioni pure**. Esse ricevono un evento in ingresso, uno Stato Corrente (rappresentato come un *Record*, ovvero una *Tuple*), eseguono una logica deterministica e restituiscono un Nuovo Stato.

Sfruttando questa proprietà, abbiamo adottato una strategia di **White-Box Testing**: invece di avviare il server in un ambiente live, i test istanziano «manualmente» strutture dati che emulano lo stato interno del manager, invocano le callback in isolamento totale e ne valutano l'output transizionale.

8.2 Implementazione della Suite EUnit (`fl_manager_tests.erl`)

Per la stesura e l'esecuzione automatizzata dei casi di test abbiamo utilizzato **EUnit**, la libreria nativa di testing unitario della BEAM VM. È stato creato il modulo dedicato `test/fl_manager_tests.erl`, la cui esecuzione è pilotabile via riga di comando (`erl -eval 'eunit:test(...)'`).

La suite copre matematicamente gli snodi decisionali più critici del `fl_manager_srv`, verificandone il comportamento mediante asserzioni strutturali (`?assertEqual`):

- **Inizializzazione Sicura:** Verifica che l'avvio del server generi un record di stato pulito (timer disarmato e nodi attesi nulli), scongiurando avvii in stati inconsistenti.
- **Calcolo Quorum:** Forzando il trigger `start_round`, il test asserisce l'effettiva allocazione di un riferimento al timer (`TimerRef /= undefined`) e il corretto calcolo dei nodi attesi (simulato a 1 nel caso di rete vuota isolata per i test).
- **Gestione del Timeout (Fallimento Totale):** Simulando uno stato fittizio in cui si attendono 3 nodi ma l'array dei pesi è vuoto, il test inietta l'evento asincrono `round_timeout`. L'asserzione garantisce che il manager abortisca l'operazione in sicurezza, ripristinando le variabili a zero senza innescare eccezioni di *pattern matching*.

8.3 Il PID Mocking: Isolamento Completo dell'Albero di Supervisione

Durante lo sviluppo della suite di test, ci siamo scontrati con un problema classico del testing nei sistemi a componenti interagenti: la dipendenza esterna. Nel test case che simulava l'accumulo con successo dell'ultimo peso atteso (il raggiungimento del quorum), il `fl_manager_srv` eseguiva il comando per cui era stato programmato: inviare i dati formattati al worker Python (`gen_server:call(python_worker_srv, ...)`).

Tuttavia, poiché il test viene eseguito in isolamento totale senza avviare l'albero di supervisione (`orchestrator_sup`), il demone Python non esisteva. Questo generava un crash `exit:{noproc}` (No Process), invalidando l'esito del test pur essendo la logica del manager intrinsecamente corretta.

Per disaccoppiare definitivamente il *Control Plane* (Erlang) dal *Data Plane* (Python) in fase di collaudo, abbiamo applicato il design pattern del **PID Mocking** (Mocking dei Processi).

All'interno del test di *Happy Path*, abbiamo istanziato dinamicamente un *Mock Process* (un attore Erlang fittizio):

```
MockPid = spawn(fun Loop() ->
    receive
        {'$gen_call', From, _Msg} ->
            gen_server:reply(From, ok),
            Loop();
        _ -> Loop()
    end
end),
register(python_worker_srv, MockPid).
```

Questo attore temporaneo si registra mascherandosi sotto l'identificativo `python_worker_srv`, agendo da «buco nero» o *dummy receiver*. Quando il manager, credendo di operare in produzione, invia il tensore matematico a Python, il *Mock* intercetta la richiesta bloccante (`$gen_call`), risponde affermativamente (`ok`) e permette al manager di chiudere regolarmente la transizione di stato senza sollevare errori. A fine test, il *Mock* viene distrutto (*Teardown*).

Questa ingegnosa tecnica ha permesso di certificare formattazioni, timing e logiche decisionali in totale isolamento, dimostrando un controllo maturo e professionale sull'infrastruttura ad attori e sulle sue dipendenze.

Capitolo 9: Conclusioni e Sviluppi Futuri

9.1 Sintesi dei Risultati Ottenuti

L'obiettivo di questo progetto era progettare e implementare un'infrastruttura distribuita, decentralizzata e *fault-tolerant*, capace di orchestrare l'addestramento di modelli di Federated Learning in uno scenario Cross-Silo (es. rete di ospedali o centri di ricerca).

I risultati ottenuti dimostrano il successo dell'approccio architetturale ibrido adottato. Affidando il *Control Plane* (orchestrazione di rete) a **Erlang/OTP** e il *Data Plane* (computazione intensiva) a **Python**, abbiamo ottenuto un sistema in cui l'inaffidabilità intrinseca del calcolo tensoriale è stata ingabbiata all'interno di un'infrastruttura resiliente.

Attraverso lo sviluppo incrementale, il progetto ha validato empiricamente i concetti cardine dei Sistemi Distribuiti:

- **Fault Isolation e Supervisione:** La comunicazione tramite flussi di Standard I/O (Ports) e l'utilizzo dell'albero di supervisione OTP (`one_for_one`) hanno garantito che crash applicativi (es. *Out of Memory*) venissero isolati e autoriparati, preservando l'integrità del nodo (Chaos Monkey Test).
- **Decentralizzazione e Consenso:** L'integrazione del demone EPMD per l'Auto-Discovery in topologia Full-Mesh e l'implementazione asincrona del *Bully Algorithm* hanno dimostrato la capacità della rete di auto-organizzarsi e stabilire una leadership temporanea senza dipendere da alcun *Single Point of Failure* (SPOF).
- **Asincronia e Tolleranza ai Guasti Parziali:** L'architettura non bloccante (*Reactor Pattern*) e l'ingegnosa gestione degli *Stragglers* tramite timeout asincroni hanno assicurato la *Liveness* del sistema, permettendo al cluster di progredire nell'aggregazione (FedAvg) pur in presenza di colli di bottiglia o partizionamenti silenti della rete («CPU Lock»).

- **Verificabilità Formale:** La validazione delle funzioni transizionali tramite White-Box Testing (EUnit) e la tecnica avanzata del *PID Mocking* hanno confermato il rigore ingegneristico dell'implementazione.

Oggi, l'architettura si presenta come un *middleware* robusto, agnostico rispetto al contenuto matematico processato e totalmente *Zero-Dependencies*, compilabile ed eseguibile nativamente.

9.2 Possibili Evoluzioni del Sistema come progetto avanzato (Sicurezza Crittografica, Integrazione PyTorch/TensorFlow)

Nonostante la stabilità dell'architettura dimostrata nei collaudi, un passaggio dall'ambiente accademico a un ecosistema di produzione reale (specialmente in ambito clinico) richiede ulteriori evoluzioni, delineando chiari sviluppi futuri come progetto avanzato.

1. Sicurezza Crittografica e Privacy (Network & Data Security):

Attualmente, il trust della rete è garantito da un *magic cookie* condiviso in chiaro. In un vero ambiente Cross-Silo, la comunicazione inter-nodo tramite EPMD deve essere blindata avvolgendo i socket TCP in tunnel TLS/SSL (sfruttando l'applicazione nativa `ssl` di Erlang). Inoltre, benché il Federated Learning protegga i dati grezzi, la condivisione dei soli pesi è soggetta ad attacchi di *Model Inversion*. Un'evoluzione cruciale consisterebbe nell'integrare algoritmi di Crittografia Omoforma (Homomorphic Encryption) o *Secure Multi-Party Computation* (SMPC), permettendo all'Aggregatore (Erlang) di eseguire o far eseguire le medie su tensori cifrati, senza mai conoscerne il valore in chiaro.

2. Integrazione con Framework ML Enterprise (PyTorch/TensorFlow):

La macchina a stati Python è stata ingegnerizzata per processare array e simulare i calcoli tensoriali. Il naturale sviluppo del *Data Plane* consisterebbe nel sostituire l'engine simulato con veri modelli in PyTorch o TensorFlow. L'architettura Erlang non richiederebbe alcuna modifica (*Location and Computation Transparency*), ma lato Python si implementerebbe un vero *training loop* sui dataset locali. In questo scenario, qualora i vettori dei pesi diventassero vettori da milioni di parametri (nell'ordine dei Gigabyte), la serializzazione testuale (JSON vanilla su Standard I/O) mostrerebbe i suoi limiti prestazionali. Lo sviluppo futuro ideale prevederebbe la migrazione del canale di comunicazione su **Protocol Buffers (Protobuf)** o su un layer locale basato su memoria condivisa (es. Apache Arrow/Plasma), mantenendo invariata la perfetta orchestrazione asincrona costruita in Erlang.